Adapting Volta to AMD GPUs: Shared Symbolic Reasoning, Target-Specific Execution

GPU Kernel Verifier contributors

Technical report — October 2026

Abstract

Volta's symbolic mathematics can be reused for native AMD kernels, but execution and synchronization need new models. We describe the gfx942, gfx950 and gfx1250 profiles: AMDHSA launch state, wave/register behavior, byte-addressed storage, matrix layouts, completion and barriers. Wave64 gfx942/gfx950 share code but differ in formats and hazards; wave32 gfx1250 requires another protocol model. Output equivalence and typed graph checks jointly determine results. This report presents implemented subsets and validation without extending Volta's formal guarantees to AMD.

1 Introduction

GPU optimizations change lane ownership, operand packing and data readiness. Porting an equivalence checker requires identifying which mathematical reasoning survives and which native semantics must change.

Original Volta symbolically executes structured CTAs with fixed control and addresses, checks synchronization, and compares output expressions over the reals. Its synchronous language has confluence, race-detection and equivalence guarantees [1]. We retain this two-stage idea. In this repository, the reusable implementation is the Rust expression DAG, specification language and canonical equivalence engine; the port does not reuse the paper's original Python/SymPy implementation unchanged.

The architecture-dependent part is substantial: native AMD assembly has an AMDHSA ABI, shared scalar state, predicated vector lanes, target-specific matrix layouts, and memory operations whose issue, completion and publication are different events. We compare the three target profiles, their execution models and independent safety checks. All are selected instruction subsets, not complete ISA models.

2 Motivating example: a shared matrix tile

Suppose one wave prepares an LDS tile and another consumes it in a matrix operation. The mathematical obligation is unchanged:

$$C'_{ij} = C_{ij} + \sum_k A_{ik} B_{kj}.$$

The native obligations are different. The following is protocol pseudocode, not a common assembly sequence for all targets:

```
producer: load operands -> wait for data -> write LDS
          -> complete LDS write -> publish through a valid barrier
consumer: participate in barrier -> read LDS -> wait for operands
          -> MFMA or WMMA -> satisfy result-readiness rules -> store C'
```

Computing the right expression is insufficient. Removing the write-completion wait must not be repaired by the barrier; removing synchronization must not be repaired by the interpreter visiting the producer first. Reusing a source or accumulator register too early is another, independent hazard. Consequently, native execution emits both output expressions and a typed event graph.

3 Background: three AMD target profiles

Table 1 summarizes what the implementation models. The public CDNA manuals, pinned LLVM semantics and independent library layouts supply the architectural references [3, 4, 5, 7, 8].

Contract	gfx942	gfx950	gfx1250
Wave/registers	Wave64; VGPR/AGPR physical allocation	Wave64; same allocation model	Wave32; no AGPR namespace
LDS capacity	64 KiB	160 KiB	320 KiB
Matrix forms	Selected F32/F16/BF16 MFMA	Adds doubled-K F16/BF16 and selected OCP FP8 MFMA	Separate F32/F16/BF16 WMMA layouts
FP8 model	FNUZ decoding not modeled	Selected OCP formats; unit-only scaled MFMA	Selected OCP conversions; FP8 WMMA not yet modeled
Completion	VM/LGKM accounting	VM/LGKM accounting	Load, Store, DS, KM, X, Async, Tensor classes
Synchronization	Workgroup barrier generations	Workgroup barrier generations	Unnamed split and allocated named shader barriers
Scalar-base index	Unsigned 32-bit	Unsigned 32-bit	Signed 32-bit before scaling

Table 1: Implemented profiles. Capacity bounds include static and dynamic LDS; instruction forms and modes have additional restrictions.

gfx942: CDNA3. The wave64 model needs explicit EXEC/VCC/SCC state, packed local IDs, physical VGPR/AGPR allocation and byte-addressed LDS/global accesses. Selected MFMA forms cover F32 16×16×4 and 32×32×2, F16 16×16×16 and 32×32×8, and the corresponding BF16 `_1k` forms. gfx942 FP8/BF8 uses FNUZ encoding; applying the gfx950 OCP decoder would change values, so that path is refused.

gfx950: CDNA4. Much of the wave64 execution can be shared, but instruction legality, numeric formats and readiness rules cannot. Selected additional MFMA forms have doubled K depth; FP8 F8F6F4 forms require a distinct interleaved input layout. The profile models selected OCP E4M3/E5M2 formats, not every F8/F6/F4 mode; scaled MFMA is limited to unit E8M0 scales. Even shared XDL MFMA forms have different delays: for selected n-pass forms, an ordinary result read/overwrite needs $n+3$ states on gfx942 and $n+4$ on gfx950; a partially overlapping or different-pass accumulator dependency needs $n+1$ versus $n+2$. Full same-pass accumulator forwarding is separate. These are dependency rules, not measured instruction latencies [6].

gfx1250. The profile changes to wave32 and low-32-bit EXEC/VCC masks. Selected WMMA forms use F32 16×16×4 or F16/BF16 16×16×32 with F32 accumulators, requiring their own lane/item maps and source lifetimes. Public gfx1250 sources also include OCP FP8 WMMA; this verifier models selected FP8 conversions, but not those matrix instructions [8]. Separate counters and replay-source retention replace the CDNA completion contract. Bounded plain 2D tensor transfers capture descriptor state and global/LDS footprints. Split and named barriers require explicit object lifetimes. None of these semantics follows by renaming a CDNA opcode.

4 Verification approach

4.1 Fixed contract and native symbolic execution

An analysis fixes the target, entry, launch dimensions, argument layout and bounded allocations. It executes workgroup $(0,0,0)$; the configured grid still supplies ABI inputs. Addresses, control decisions and synchronization parameters must become concrete, while arithmetic data may remain symbolic.

Native parsing retains descriptors, argument offsets, globals, modifiers and source spans. Typed lowering checks legality again for direct Rust callers. AMDHSA metadata determines register assignment and kernarg padding. Unavailable runtime fields remain undefined: enabling a Dispatch ID slot does not justify initializing it to zero [5].

The executor tracks one PC and scalar register file per wave, lane-indexed vector values, masks, byte-defined global/LDS storage, pending operations and barrier state. Partial loads and narrow updates preserve defined bits without inventing the rest. Concrete integer/address operations retain width, signedness and wrapping; encoded F16/BF16/FP8 paths are distinct from abstract real values. Optional named LDS subobjects constrain complete transfers within aggregate LDS.

For each matrix instruction, target maps translate lane/register items to A_{ik}, B_{kj} and C_{ij}, then construct the expression in Section 2. Matrix legality, full-wave participation and encoded formats are checked before that computation. A shared multiplication formula therefore does not imply a shared fragment layout.

4.2 Separate execution from safety

The executor may calculate asynchronous values eagerly to build expressions. It also records issue events, byte effects, counter charges, physical register accesses, source leases and synchronization state. A target-selected checker reconstructs four obligations:

1. **Trace completeness:** instructions, operands and semantic records are consistent with the target; missing records cannot establish safety.
2. **Readiness:** pending destinations cannot be consumed, and retained sources cannot be overwritten before the relevant wait or dependency delay.
3. **Barriers:** participants, generations, counts and lifetimes justify release and progress.
4. **Memory conflicts:** overlapping effects with a write have the required completion, publication and ordering.

The repository's graph infrastructure is shared with PTX. Its PTX tensor/proxy certificates are not reused as AMD safety rules. Graph checking validates typed records; it trusts the producer's instruction values and memory addresses and is not a second assembly interpreter.

CDNA completion and publication. gfx942/gfx950 waits discharge VM/LGKM obligations; FLAT can charge both. Nonzero thresholds certify an ordered prefix only where the modeled class permits it. A workgroup barrier publishes completed effects; it does not complete pending memory. The selected cross-wave GLOBAL rule additionally requires workgroup scope at both endpoints and the explicit single-CU workgroup premise.

gfx1250 counters, replay and barriers. Data completion and X translation/source retention are tracked separately: draining X does not make load data ready, while a data-completion wait also retires that operation's translation lease. Tensor waits currently require zero. Cross-wave write-dependency publication for ordinary GLOBAL/resolved-global FLAT graph events requires explicit system scope, completion and checked ordering; the source profile exposes that scope modifier only for resolved-global FLAT. These rules do not establish inter-workgroup safety [7].

Unnamed barrier arrivals that participate in a wait need a completing wait execution-before the signaller's implicit END drop. Named barriers instead track physical ID, initialization incarnation, phase, joins/leaves and expected count. Selected nonzero M0 signal-time updates change the expected count before arrival; END does not drop named objects. Ordering cannot be inferred merely from candidate trace order, and ambiguous public count boundaries stay inconclusive.

4.3 Shared equivalence and the guarantee boundary

Native outputs use the existing expression DAG and canonicalizer, with exact coefficients, shared subexpressions and memoization. For each checked output array a, let I_a be its required indices and W_a its written indices. The implementation's acceptance contract is

$$\textsc{Accept} \iff \text{all analyzed kernels complete and all applicable checks pass,}$$
$$\forall a:\ I_a = W_a\ \wedge\ \forall i \in I_a:\ O_a(i) \equiv R_a(i).$$

This describes the tool's decision rule, not a hardware soundness theorem. Footprints are checked before sampling; duplicate definitions and overflowing shapes are rejected. Complete verification also requires checked elements to equal the full total. A refutation rejects; unsupported semantics, resource limits or unproved identities remain inconclusive absent another refutation. The CLI combines results; Python exposes separate graph, equivalence and count fields. Expression replay alone carries no execution-safety result.

Original Volta's confluence argument is for its specified synchronous language [1]. Extending that argument requires native operational semantics, completion/publication rules and a proof connecting them to this graph checker. Those proofs are not supplied here. Real-arithmetic equality also does not establish IEEE rounding, NaN behavior or approximation error.

5 Implementation and evaluation

The implementation snapshot is `fd640e6`; the PTX-only comparison baseline is `e8f8199`. The main code paths are native parsing/execution, CDNA graph checking, gfx1250 graph checking, equivalence and result aggregation. Public semantic revisions are pinned in the reference manifest. Checked-in assembly can be analyzed on a CPU without ROCm or a device.

Validation separates questions that a single pass count cannot answer:

- Compiler/assembler fixtures check exact emitted forms and ABI layout. Cross-target controls reject relabeled instructions and incompatible encodings.
- Basis/asymmetric matrix tests check fragment coordinates; native mutations check missing waits, early reuse, malformed descriptors and barrier lifetimes.
- Full output contracts check dimensions, storage, projections and every required index; independent numerical drivers check finite device behavior.

At the stated snapshot, the release workspace passed 2,089 Rust tests and the rebuilt extension passed 113 Python tests. These are regression results, not an ISA coverage percentage or a proof of all schedules. The original-workload registry contains 32 registrations; default CLI/Python interface tests select W01–W18. Static registry consistency is not evidence that every full-size workload was freshly executed. No new GPU run is claimed in this report.

Reproduction starts with `cargo test -release -workspace` and `just test-python`; the usage guide gives a small complete native check, while full-workload drivers document resource requirements and all-workload selection. Unsafe synchronization mutations stay in CPU analysis.

6 Related work and limitations

AI as a Compiler preserves a fixed specialization/ABI/target contract while generating PTX. Its Volta extensions add asynchronous movement, modern tensor operations, packed values and broader synchronization; Appendix B explicitly leaves soundness and completeness unclaimed [2]. The common lesson is that mathematical outputs and native ordering both need models. Blackwell tensor memory, commits and proxy fences are not AMD WMMA, completion counters or shader-barrier objects; their rules cannot be transferred by analogy.

The AMD implementation remains an experimental bug-finding tool for configured native subsets. It does not establish whole-grid or all-scheduling correctness, arbitrary data-dependent control, general atomic interleavings or full IEEE semantics. gfx942 FNUZ decoding, arbitrary symbolic BF16 reinterpretation, nonunit scaled MFMA, higher banked gfx1250 register encodings and advanced tensor or clustered modes remain outside the selected profiles. Unsupported cases must stay visible. Broader differential tests and a formal native-to-graph account are the next steps toward stronger guarantees.

7 Conclusion

Volta's symbolic-output comparison remains useful across all three AMD targets. The required changes lie in launch/ABI state, wave execution, storage formats, matrix layouts and target-specific completion and synchronization. Sharing the algebra engine avoids rebuilding equivalence reasoning; separating native safety contracts avoids treating similar-looking GPU instructions as equivalent semantics.

References

[1] K. Dubey, B. Driscoll, A. Wei, N. Kayal, R. Sharma and A. Aiken. *Equivalence Checking of ML GPU Kernels*. 2025. arXiv:2511.12638v1.

[2] F. Costa, C. Castes, T. Bourgeat and A. Mirhoseini. *AI as a Compiler: Compiling Triton kernels without the Triton compiler*. 2026. arXiv:2609.36800v1, Section 4.5 and Appendix B.

[3] AMD. *AMD Instinct MI300 Instruction Set Architecture*. Public CDNA3 ISA manual.

[4] AMD. *AMD CDNA4 Instruction Set Architecture Reference Guide*. Public CDNA4 ISA manual.

[5] LLVM/ROCm contributors. *User Guide for AMDGPU Backend*. Revision `5fd50ec8`. Pinned ABI and memory-model reference.

[6] LLVM contributors. *GCNHazardRecognizer.cpp*, LLVM 22.1.0, revision `4434dabb`. Pinned MFMA dependency rules.

[7] LLVM/ROCm contributors. Revision `5fd50ec8`: *AMDGPU Asynchronous Operations*, *AMDGPU DMA Operations*, and *AMDGPU Execution Synchronization*.

[8] ROCm FlyDSL contributors. Revision `f37f7a2b`: *WmmaLayout.h* and *MmaAtom.cpp*.